JavaScript (Parte 3 y 4)

Desarrollo de Aplicaciones en Entornos Web Curso 2024/2025

Contenido

- Parte I: Introducción a **JavaScript**
- Parte II: **Expresiones Regulares** en JavaScript
- Parte III: Api Browsers en JS : **DOM** y **BOM**
- Parte IV: Modelo de **Eventos**

Parte III: API DOM y BOM

- *JavaScript* puede acceder al contenido de una página utilizando un API denominada **DOM** (Doument Object Model) e interaccionar con los navegadores con el API **BOM** (Browser Object Model).

- El API DOM ha sido **estandarizada** por el consorcio W3C.

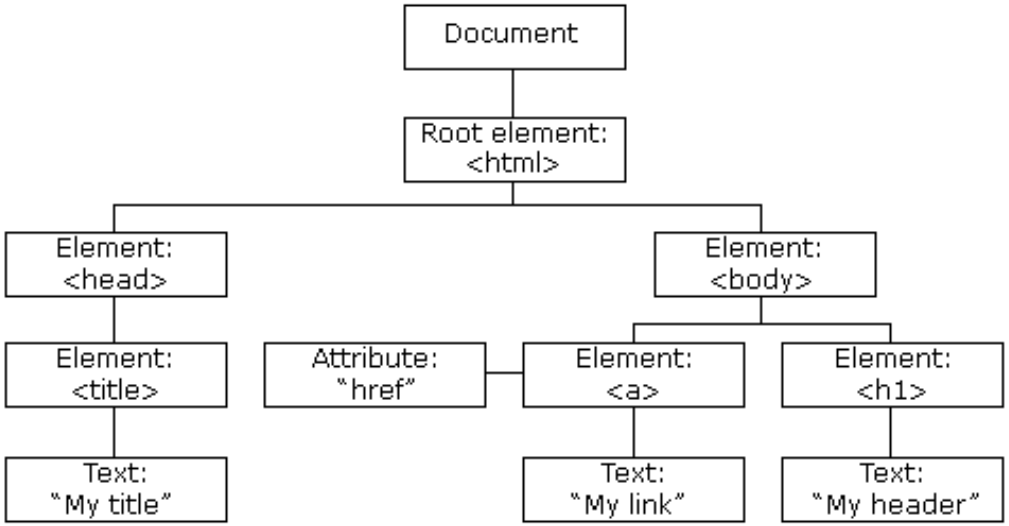
  https://www.w3.org/DOM/

  https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

  - DOM ofrece varios modos de acceso al contenido de una página: desde nivel 0 a nivel 3
  - A través del objeto `document`
  - Mediante un árbol de objetos/nodos
  - Selectores de Nodos

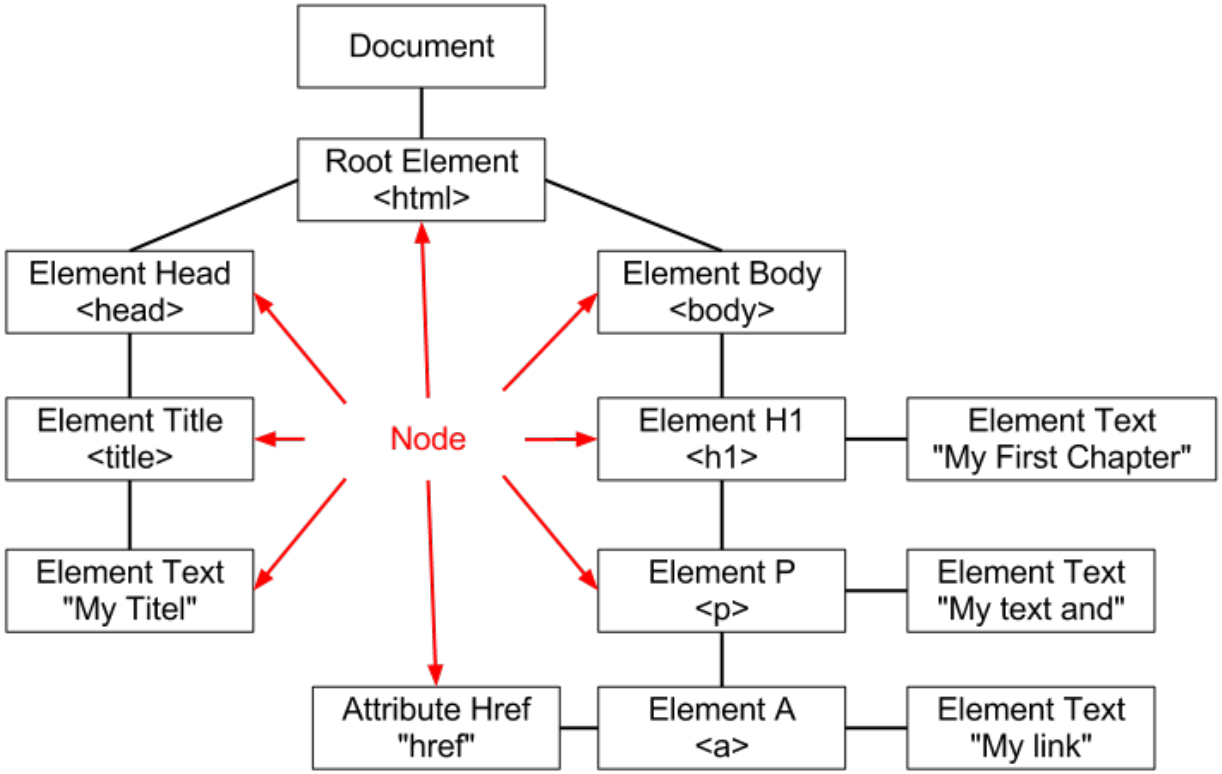
- DOM no es parte de la especificación de JavaScript, es una API para los navegadores.

- **DOM**: representación estructural del documento, permitiendo modificar su contenido y presentación visual mediante JS.



DOM - document

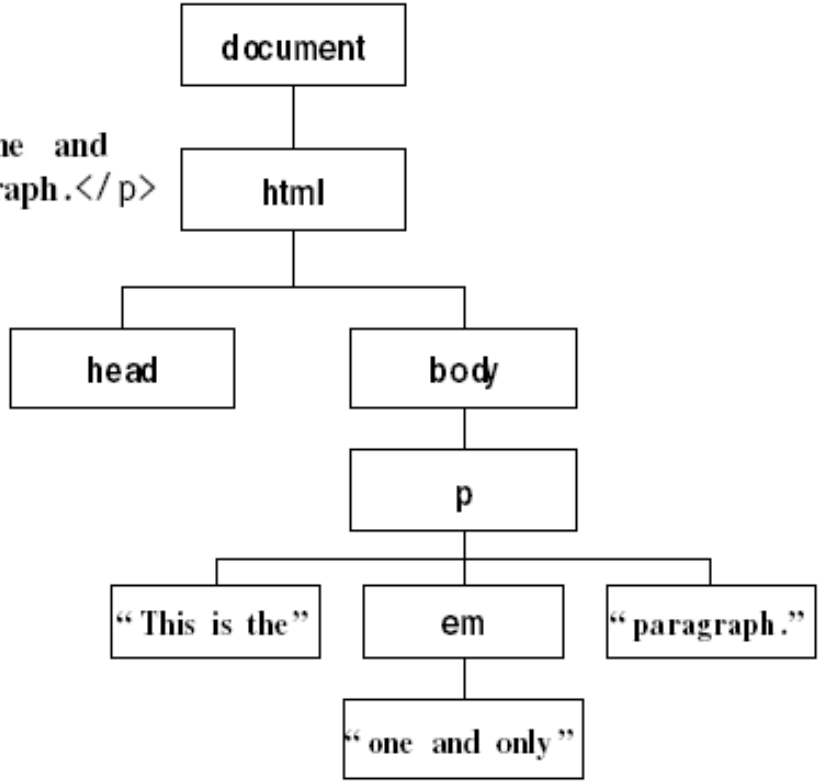
- **document** representa la **página HTML**
- Permite el acceso a **toda** la información del documento.



DOM nivel 1 CORE

- **DOM Nivel 1**: el navegador crea un **árbol** de nodos que representa los elementos de la página HTML

```
<html>
   <head></head>
   <body>
        <p>This is the <em>one and
             only </em> paragraph.</p>
   </body>
</html>
```



- Disponible accediendo a los elementos document o window, para manipular el documento mismo o sus diferentes partes

```
<html>
  <head>
    <script>
       // ejecuta esta función cuando se cargue el documento
       window.onload = function() {
         // crea dinámicamente un par de elementos HTML en una página vacia
         var heading = document.createElement("h1");
         var heading_text = document.createTextNode("el texto que desee");
         heading.appendChild(heading_text);
         document.body.appendChild(heading);
       }
    </script>
  </head>
  <body>
  </body>
</html>
```

```
console.log(window.document);
console.log(document);
console.log(document.head);
console.log(document.body);
console.log(document.documentElement);
console.log(document.doctype);
console.log(document.charset);
console.log(document.title);
console.log(document.links);
console.log(document.images);
console.log(document.forms);
console.log(document.styleSheets);
console.log(document.scripts);
setTimeout(() => {
  console.log(document.getSelection().toString());
}, 2000);
document.write("<h2>Hola Mundo desde el DOM</h2>");
```

Referencias a nodos del documento

- Obtener referencias a declaraciones por el atributo **id.**

```
var logo = document.getElementById("logo");
```

- Obtener todos los objetos de un **tipo de etiqueta**:
  - Ejemplo: obtener todos los bloques *div*

```
var array = document.getElementsByTagName("div");
```

  - También se puede aplicar sobre cualquier nodo del árbol DOM.

- Obtener referencias a declaraciones por el atributo **name.**

```
var logo = document.getElementByName("un_boton");
```

Nodos, Elementos y Selectores

```
console.log(document.getElementsByTagName("li"));
console.log(document.getElementsByClassName("card"));
console.log(document.getElementsByName("nombre"));
console.log(document.getElementById("menu"));

console.log(document.querySelector("#menu"));
console.log(document.querySelector("a"));
console.log(document.querySelectorAll("a"));
document.querySelectorAll("a").forEach((el) => console.log(el));

console.log(document.querySelector(".card"));
console.log(document.querySelectorAll(".card"));
console.log(document.querySelectorAll(".card")[2]);
console.log(document.querySelector("#menu li"));
console.log(document.querySelectorAll("#menu li"));
```

Atributos y Data-Attributes

```
console.log(document.documentElement.lang);
console.log(document.documentElement.getAttribute("lang"));
console.log(document.querySelector(".link-dom").href);
console.log(document.querySelector(".link-dom").getAttribute("href"));

document.documentElement.lang = "en";
document.documentElement.setAttribute("lang", "es-MX");

const linkDOM = document.querySelector(".link-dom");

linkDOM.setAttribute("target", "_blank");
linkDOM.setAttribute("href", "https://www.um.es");

console.log(linkDOM.hasAttribute("rel"));

linkDOM.removeAttribute("rel");
```

Estilos y Variables CSS

```
const linkDOM = document.querySelector(".link-dom");

console.log(linkDOM.style);
console.log(linkDOM.getAttribute("style"));
console.log(linkDOM.style.backgroundColor);
console.log(linkDOM.style.color);

linkDOM.style.setProperty("text-decoration", "none");
linkDOM.style.setProperty("display", "block");
linkDOM.style.width = "50%";
linkDOM.style.textAlign = "center";
```

Clases CSS

```
const card = document.querySelector(".card");

console.log(card.className);
console.log(card.classList);
console.log(card.classList.contains("rotate-45"));
card.classList.add("rotate-45");
card.classList.remove("rotate-45");
card.classList.toggle("rotate-45");
card.classList.replace("rotate-45", "rotate-135");
```

Texto y HTML

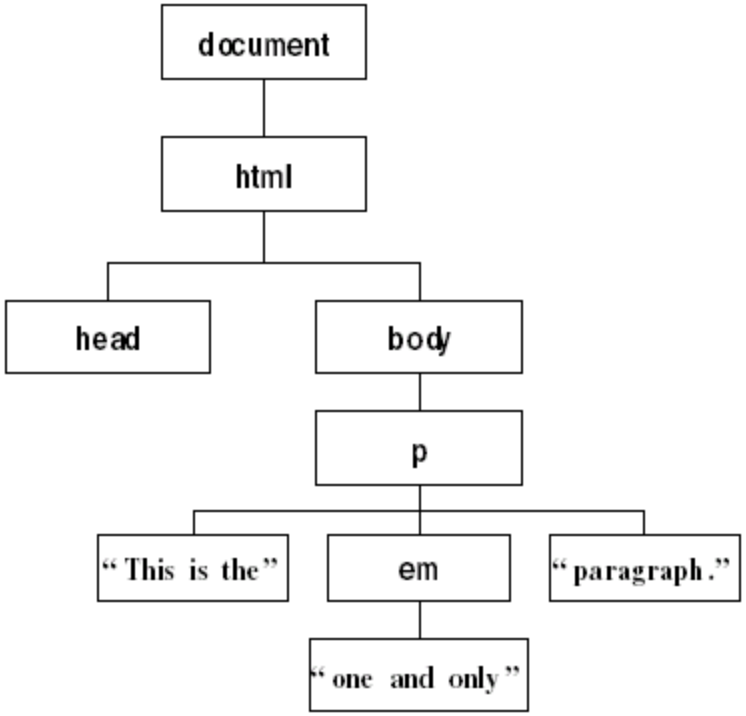
```
const whatIsDOM = document.getElementById("que-es");
let text = '
    <p>
      El Modelo de Objetos del Documento (<b><i>DOM - Document
Object Model</i></b>) es un API para documentos HTML y XML.
    </p>';

whatIsDOM.innerHTML = text;
whatIsDOM.outerHTML = text;
```

Recorrido de los nodos

- **Tipos de nodos**:
  - Nodos de **elementos (etiquetas)**
  - Nodos de **texto**.
  - Raíz: **document**.

- **Propiedades** de los nodos:
  - `firstChild, lastChild, childNodes[],`
  - `parentNode, nextSibling, previousSibling`



Recorrido de los nodos (Node)

```
var objeto_html = document.documentElement;
```

- Donde objeto_html es un nodo *HTMLElement*

```
var objeto_head = objeto_html.firstChild;
var objeto_body = objeto_html.lastChild;

var objeto_head = objeto_html.childNodes[0];
var objeto_body = objeto_html.childNodes[1];

var objeto_head = document.head;
var objeto_body = document.body;
```

Recorrido de los nodos (Element)

```
const cards = document.querySelector(".cards");

console.log(cards.children);
console.log(cards.children[2]);
console.log(cards.parentElement);
console.log(cards.firstElementChild);
console.log(cards.lastElementChild);
console.log(cards.previousElementSibling);
console.log(cards.nextElementSibling);
console.log(cards.closest("div"));
console.log(cards.children[3].closest("section"));
```

Tipos de Datos

- document
  - Raíz del objeto documento en sí mismo.
    - por ejemplo, la propiedad `ownerDocument` de un elemento devuelve el documento "document" al cual pertenece
- element
  - por ejemplo, `document.createElement()`
- attribute
  - por ejemplo, `document.createAttribute()`
- **nodeList**
  - Por ejemplo, `document.getElementsByTagName()`.
  - Acceso a los items de la nodeList: `list.item (1), lista [1]`
  - `Array.from(list) + array.forEach()`

Ejemplo

```
var table = document.getElementById("table");
var tableAttrs = table.attributes; // Node/interfaz Element

for (var i = 0; i < tableAttrs.length; i++) {
  // interfaz HTMLTableElement: atributo border
  if(tableAttrs[i].nodeName.toLowerCase() == "border")
    table.border = "1";
}

// interfaz HTMLTableElement: atributo summary
table.summary = "nota: borde aumentado";
```

querySelector – document element

```
element = document.querySelector(selectores);
```

- Devuelve el **primer Element** del documento que coincida con el grupo especificado de selectores.
  - utilizando un recorrido primero en profundidad pre ordenado de los nodos del documento
  - **selectores** es una cadena de caracteres que contiene uno o más selectores CSS separados por coma

```
var el = document.querySelector(".miClase");
var el = document.querySelector("div.user-panel input[name='login']");
```

- el primer elemento <input name="login" /> dentro de .

querySelectorAll – document element

```
elements = document.querySelectorAll(selectores);
```

- Retorna una **NodeList** ***no viva*** que contiene un objeto **Element** para cada elemento que coincida con al menos uno de los selectores especificados
  - o una NodeList vacía en caso de que no haya coincidencias
  - **SyntaxError**: La sintaxis de la cadena de texto *selectors* no es válida.

```
var matches = document.querySelectorAll('div.nota,
    div.alerta');
```

- Para convertir una NodeList en un Array:

```
let nodeList = document.querySelectorAll(selector);
let elements = Array.from(nodeList);
```

- NodeList es un tipo de objeto Array, pero NO un array.
  - Sin embargo, algunos navegadores web permiten usar **`forEach()`** o **`for...of`**.

Creación de nodos

- document.createElement
- document.createTextNode

```
var newDiv = document.createElement("div");
var newContent = document.createTextNode("Hola!¿Qué tal?");
newDiv.appendChild(newContent);             //añade texto al div creado.
// añade el elemento creado y su contenido al DOM
var currentDiv = document.getElementById("div1");
document.body.insertBefore(newDiv, currentDiv);
```

- document.createAttribute

```
var nodo = document.getElementById("div1");
var a = document.createAttribute("miAtributo");
a.value = "nuevoVal";
nodo.setAttributeNode(a);
```

Manipulación de atributos

- Los nodos de tipo **Element** contienen la propiedad **attributes**
  - getAttribute(nombre), setAttribute(nombre, valor), removeAttribute(nombre), hasAttribute(nombre)

```
<p id="introduccion" style="color: blue">
   Párrafo de prueba
</p>

var p = document.getElementById("introduccion");
var elId = p.getAttribute("id");
p.setAttribute("id", "preintroduccion");
```

Ejemplo creación con DOM

```
const figure = document.createElement("figure"),
img = document.createElement("img"),
figcaption = document.createElement("figcaption"),
figcaptionText = document.createTextNode("Animals"),
cards = document.querySelector(".cards"),

img.setAttribute("src", "https://placeimg.com/200/200/animals");
img.setAttribute("alt", "Animals");
figure.classList.add("card");

figcaption.appendChild(figcaptionText);
figure.appendChild(img);
figure.appendChild(figcaption);
cards.appendChild(figure);
```

Modificación (Node)

- **Modificación de los hijos** desde el nodo padre:
  - `insertBefore(nuevoHijo, nodoHijoReferencia)`
  - `replaceChild(nuevoHijo, antiguoHijo)`
  - `appendChild(nuevoHijo)` → añade al final
  - `removeChild(hijo)`

  - Si el nodo hijo es una referencia a un nodo ya existente, entonces lo mueve de la posición actual a la nueva posición.

- Los **atributos de HTML** son accesibles mediante propiedades:
  - Ejemplo: `imagen.src = "imagen.gif";`

Modificando Elementos (old way)

```
const cards = document.querySelector(".cards"),
  newCard = document.createElement("figure"),
  cloneCards = cards.cloneNode(true);

newCard.innerHTML = `
  <img src="https://placeimg.com/200/200/any" alt="Any">
  <figcaption>Any</figcaption>’;
newCard.classList.add("card");

//cards.replaceChild(newCard, cards.children[2]);
//cards.removeChild(cards.lastElementChild);
cards.insertBefore(newCard, cards.firstElementChild);
document.body.appendChild(cloneCards);
```

InsertAdjacentHTML (cool way)

- Parsea el texto pasado como parámetro y lo inserta dentro del DOM en una posición específica.

```
element.insertAdjacentHTML(position, html);
```

  - **`position`**
    - **beforebegin**: Antes que el propio elemento.
    - **afterbegin**: Justo dentro del elemento, antes de su primer elemento hijo.
    - **beforeend**: Justo dentro del elemento, después de su último elemento hijo.
    - **afterend**: Después del propio elemento.
  - **`html`**: contenido HTML a insertar.

```
var lista;
var boton = document.getElementById("add");

boton.addEventListener("click",function(){
   lista = document.getElementById("lista");
   lista.insertAdjacentHTML(  "beforeend",
                              "<li>Elemento</li>");
},false);
```

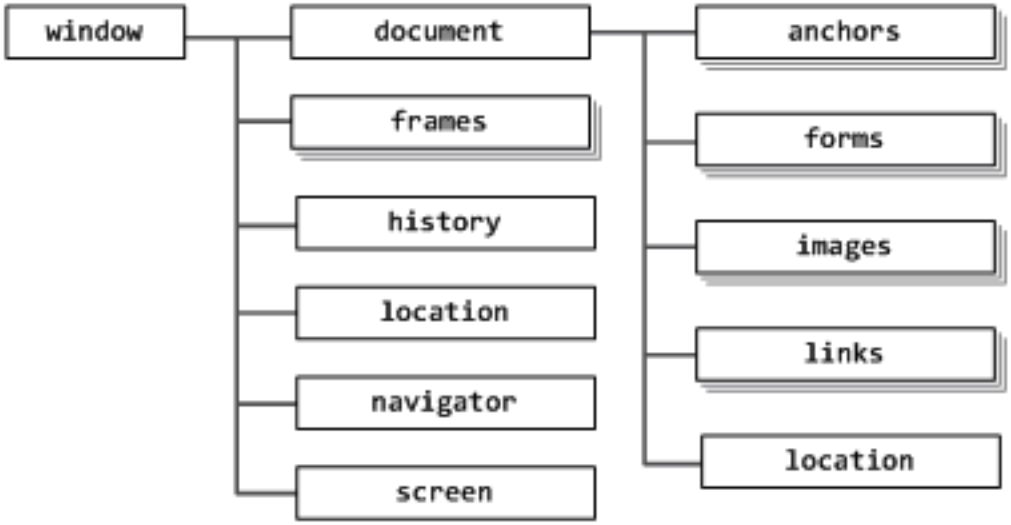
Modificando Elementos (cool way)

```
/*
.insertAdjacent...
  .insertAdjacentElement(position, el)
  .insertAdjacentHTML(position, html)
  .insertAdjacentText(position, text)
Posiciones:
  beforebegin(hermano anterior)
  afterbegin(primer hijo)
  beforeend(ultimo hijo)
  afterend(hermano siguiente)
*/
const cards = document.querySelector(".cards"),
  newCard = document.createElement("figure");
let contenCard = '<img src="https://placeimg.com/200/200/any" alt="Any">
  <figcaption></figcaption>';
newCard.classList.add("card");

newCard.insertAdjacentHTML("afterbegin", contenCard);
cards.insertAdjacentElement("beforeend", newCard);
newCard.querySelector("figcaption").insertAdjacentText("afterbegin", "Any"); */
```

BOM - Objetos del navegador

- El código *JavaScript* puede acceder al contenido de un navegador utilizando el API denominada **BOM** (Browser Object Model).



BOM - window

- **window** ofrece el **contexto general del navegador**

- [deprecated] Si una página emplea frames, cada uno de ellos se almacena en el array **frames**

```
window.frames[0]; window.frames["nombre del frame"];
```

- BOM define cuatro métodos para manipular el tamaño y la posición de la ventana:
  - **moveBy**(x, y) basado en incrementos, ***moveTo****(x, y)* basado en posiciones absolutas, ***resizeBy****(x, y)*, ***resizeTo****(x, y)*
  - Navegadores **poco permisivos** en la modificación de la ventana

BOM - window

- Abrir **diálogos**:
  - `window.alert("…")` ó `alert("…")` → aviso
  - `window.confirm("…")` ó `confirm("…")` → devuelve *true* o *false*

- Abrir **diálogos confirmación**:
  - `window.prompt("…")` ó `prompt("…")`

- **Propiedad location**:
  - Cambia la página que visualiza la ventana.
  - `window.location.reload()`
  - `window.location.`***`href`*** `= http://www.um.es`
  - Otras propiedades: host, pathname (contenido después del host), port, protocol, search (contenido después de '?')
- **Propiedad status***:* acceso a la barra de estado
- **Propiedad console***:* acceso a la salida consola.
  - `console.log()`
  - `console.error()`

BOM - window

- **Propiedad navigator**:
  - Propiedad **userAgent*:*** cadena que identifica al navegador.
  - Otras propiedades: browserLanguage/language, cookieEnabled

- **Propiedad history**:
  - Historial de navegación de la ventana.
  - Métodos ***back***() y ***forward***() equivalen a los botones “Atrás” y “Adelante”.
  - Método ***go(number)*** Ej: *history.go(-2)* se mueve dos páginas atrás en el historial

- **Propiedad screen** (pantalla):
  - Acceso a las dimensiones de la pantalla.
  - Propiedades: ***width, height, availWidth, availHeight***

```
window.moveTo(0, 0);
window.resizeTo(screen.availWidth, screen.availHeight);
```

BOM - document

- **document** es el único objeto que pertenece tanto a DOM como a BOM.

- Propiedades simples:
  - lastModified : Fecha de la última modificación
  - referrer : URL desde la que se accedió
  - title : etiqueta <title>
  - URL : URL de la página actual
- Algunas propiedades arrays (acceso simple o asociativo):
  - forms : todos los formularios de la página
  - images : todas las imágenes de la página
  - links : todos los enlaces de la página (<a href>)

Formularios

- **Acceso**:
  - `document.forms["nombreFormulario"]`
  - Alternativa: `document.nombreFormulario`
- Acceso a los **controles de un formulario:**
  - Array **elements**
  - `formulario.elements["nif"]`
  - Alternativa: `formulario.nif`
- La acción de los **botones** de tipo *submit* y *reset* puede ser realizada aplicando métodos del formulario:
  - `formulario.submit()`
  - `formulario.reset()`

Formularios - Controles

- **Controles de tipo input:**
  - Están disponibles los **métodos**:
    - **focus**(): ganar el foco de control
    - **blur**(): sacar el foco de control
    - **select**(): si contiene texto, lo marca seleccionado
- **Controles tipo select**: (menús y listas)
  - Opciones accesibles a través del array **options**
  - Propiedad **selectedIndex:**
    - Almacena el índice de la opción seleccionada.
    - Si su valor es -1, no hay ninguno seleccionado.

Formularios - Controles

- **Botones de radio**:
  - El control es manejado como un **array.**
  - Propiedad **length***:* número de botones.

- Todos los **atributos de HTML** son accesibles como propiedades:

```
formulario.elements["nif"].value;
```

- Los **atributos vacíos** (ejemplo *checked*) son de tipo booleano:

```
formulario.elements["licencia"].checked = false;
```

Acceso a propiedades de estilo

- Estilos de una etiqueta accesibles mediante **propiedad style**:

```
document.getElementById("intro").style.color = "red";
document.getElementById("intro")getAttributte('style')
```

- **Precaución con nombres de las propiedades de estilo**:
  - **Guiones** → cambiar a mayúscula. Ej. `font-size` a `fontSize`
  - Los valores de TODAS las propiedades son cadenas, incluso las medidas: ejemplo "1.5"

- Consultar y cambiar la **clase** de una etiqueta:
  - Propiedad **className/classList.**

Eventos (old way)

- **Eventos**:
  - Introducidos en la versión 4 del estándar HTML
  - Ejemplos eventos:
    - Acciones del usuario sobre algún elemento de la página: click, doble click, modificar un campo, etc.
    - Cambio estado interno de la página: la página está disponible, se abandona la página web.
- **Los eventos son tratados con funciones (manejadores).**
- **Registrar un manejador de evento**:
  - Dentro de la etiqueta:
    - `<button ... onclick="manejador()">`
  - Asignando en JS la referencia a la función:
    - `boton.onclick = manejador;`

Eventos (old way)

- **Acción por defecto**:
  - Algunos eventos tienen asociada una acción por defecto. Ejemplo: pulsar el botón *submit* de un formulario.
  - Es posible controlar si se permite la acción por defecto.
    - `<input ... onchange="return manejador()">`
  - Para anular la acción por defecto el manejador devuelve `false`.
- Establecer en el manejador el nodo que recibe el evento utilizando **this**:
  - `<input ... onchange="manejador(this)">`
    - **`this`** referencia al nodo. En el manejador el parámetro se puede utilizar para acceder a las propiedades del nodo.

Manejadores de Eventos

- Utilizar los atributos XHTML para añadir manejadores *"**ensucia**" el código XHTML*

```
<input id="pinchable" type="button"
   value="Pinchame y verás"
   onclick="alert('Gracias por pinchar');" />
```

- Manejadores separados: el código XHTML es *"**limpio**" pues no se mezcla con el código JavaScript.*

Manejadores de Eventos

```
function muestraMensaje() {
   alert('Gracias por pinchar');
}
document.getElementById("clickable").onclick =
                                        muestraMensaje;
<input id="clickable" type="button" value="Hacer
click" />
```

- Asignar identificador único al elemento XHTML mediante el atributo **id**.
- Crear una función de JavaScript encargada de manejar el evento.
- Asignar la función a un evento concreto del elemento XHTML mediante DOM.

addEventListener (**current way**)

- **addEventListener**: registra un listener de eventos, como se especifica en W3C DOM.
  - Permite agregar mas de un listener a un solo evento.
  - Funciona en cualquier elemento del DOM, no únicamente con elementos de HTML.
    - El objeto puede ser un simple *elemento* en un archivo, el mismo *documento* , una *ventana* o un *XMLHttpRequest*.

```
var el = document.getElementById("mi_tabla");
el.addEventListener("click", modifyText);

function modifyText(e) {
    …
    e.preventDefault()
}
```

addEventListener

- evento.**preventDefault()**: evita que se ejecute la acción por defecto

```
addEventListener('click',
    function(e) {
      ...
      e.preventDefault();
      ...
      }
```

- Información del evento:

```
e.target: quien produjo el evento
e.type: tipo de Evento. Ej: 'click'
```

Tipos de Eventos

https://w3c.github.io/uievents/#event-types-list

- **Foco de control y cambios** (sólo controles de formularios):
  - blur: pérdida foco de control
  - focus: gana el foco
  - change: contenido ha cambiado y pérdida del foco
  - select: selección texto.
- **Ratón**: sobre cualquier elemento
  - click: clic sobre un elemento
  - dblclick: doble clic
  - mousedown: pulsar el ratón
  - mouseup: liberar pulsación ratón
  - mouseover: entrar puntero zona elemento
  - mouseout: salir con el puntero de la zona del elemento

Tipos de Eventos

- **Teclado**:
  - keypress: pulsación tecla (completa)
  - keydown: tecla pulsada (no liberada)
  - keyup: liberación tecla
  - cut/copy/paste: cortar, copiar/pegar
- **Formularios:**
  - submit: pulsación botón envío formulario
  - reset: inicializar/resetear formulario
- **Carga de la página (`<body>`):**
  - load: carga del documento → documento accesible
  - unload: un documento va a ser descargado, sale de la página

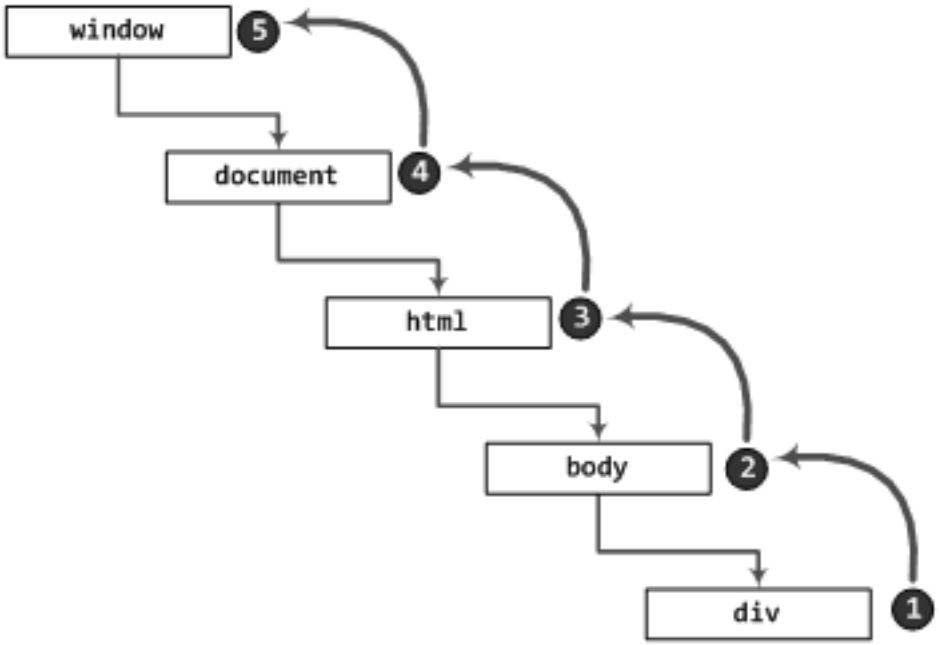
Flujo de Eventos

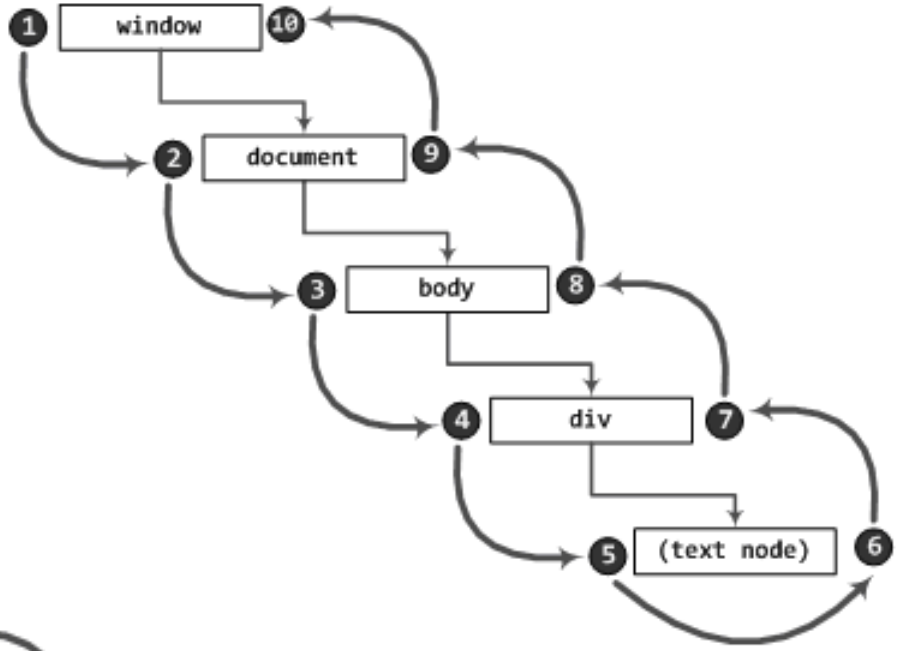
- Flujo de eventos: El orden en el que se ejecutan los eventos asignados a cada elemento de la página
  - Permite que que varios elementos diferentes puedan responder a un mismo evento.

- Modelos de flujo:
  - **Event Bubbling**: desde el elemento más específico hasta el elemento menos específico (más común)
  - **Event Capturing**: desde el elemento menos específico hasta el elemento más específico (contrario al *"event bubbling”).*
  - **Event DOM**: definido en la especificación **DOM**. Soporta tanto el *bubbling como el capturing.*
    - Los dos flujos de eventos recorren todos los objetos DOM desde *document* hasta el elemento más específico y viceversa, continuando el flujo hasta el objeto *window*.

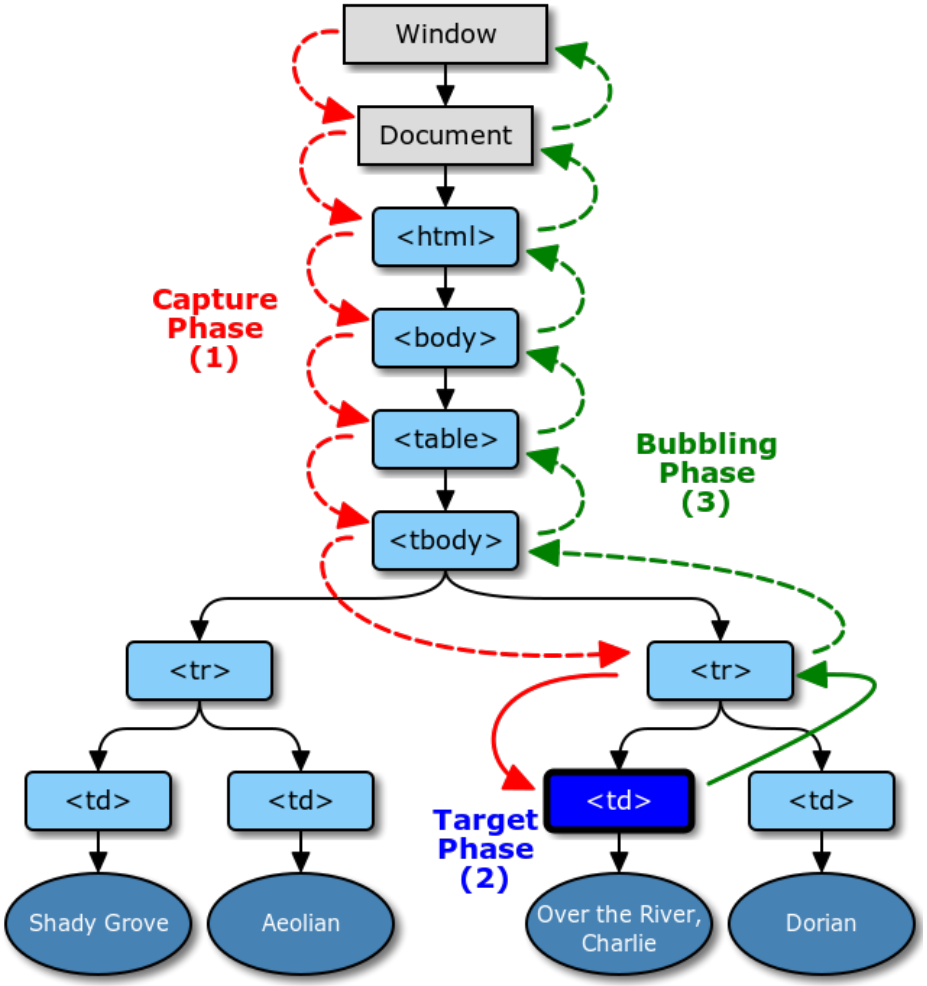
Flujos de Eventos

Event Bubbling





Event DOM


Window
Document
<html>
<body>
<table>
<tbody>
<tr>
<tr>
<td>
<td>
<td>
<td>
Shady Grove
Aeolian
Over the River, Charlie
Dorian
Capture Phase (1)
Target Phase (2)
Bubbling Phase (3)

```
var el = document.getElementById("mi_tabla");
el.addEventListener("click", modifyText, false);
```

- boolean **useCapture** :
  - Indica si el evento se propaga/está activado durante la fase de captura.
  - Por defecto `false`, NO se aplica fase *capturing*

- El evento se propaga a TODOS los elementos durante la propagación.
  - Para detener la propagación: `evento.stopImmediatePropagation()`

```
let ele = document.querySelector('#mi_enlace');
ele.addEventListener('click', handlingClick);

function handlingClick(event) {
        ...event.stopPropagation()...
        ...event.preventDefault()...
}
```

- El método **stopPropagation()** de la interfaz ***Event*** evita la propagación adicional del evento actual en las fases de *captura* y *bubbling*.
  - Sin embargo, no previene de la ejecución de las acciones por defecto (p.e. click en un enlace, producirá navegación), salvo que se use *event.preventDefault()*)

- El método **preventDefault()** cancela el evento (previene la ejecución por defecto) si este es cancelable. Pero el evento continúa propagándose (salvo que se use *event.stopPropagation()*)

Ejemplo: Mostrar y ocultar bloques

- **Uno de los efectos más útiles** en la programación dinámica HTML es mostrar y ocultar un bloque.
- Normalmente, se define una función del siguiente modo:
  - Parámetro: **"id"** del elemento sobre el que actúa.
  - Utiliza la propiedad de **estilo "display"**.
  - El bloque debe tener establecido explícitamente la propiedad de estilo display para que funcione la función.

```
function muestraBloque(id) {
    var bloque = document.getElementById(id);
    if (bloque.style.display == "none")
        bloque.style.display = "block";
    else block.style.display = "none";
}
```

LocalStorage & SessionStorage

- El objeto **Storage** nos permite almacenar datos de manera local en el navegador
  - sin necesidad de realizar alguna conexión a una base de datos
- **localStorage** y **sessionStorage** son propiedades que acceden al objeto Storage para almacenar datos de manera local
- **localStorage** almacena la información de forma indefinida o hasta que se decida limpiar los datos del navegador
- **sessionStorage** almacena información en la sesión web actual

- Ambas se **manejan igual**:

```
localStorage.setItem('clave', 'valor')
let valor = localStorage.getItem('clave')
localStorage.removeItem('clave')
if (localStorage.length > 0 )
  localStorage.clear()
```

Trabajando con JSON en JS

- **JSON:** objeto del API para trabajar con contenido en formato JSON
  - disponible en todos los navegadores modernos

- **JSON.stringify()**: toma un objeto de JavaScript y lo transforma en una cadena JSON.
  - Trabaja también con objetos Arrays.
- **JSON.parse()**: toma una cadena JSON y la transforma en un objeto de JavaScript

```
const myObj = {
  name: 'Skip',
  age: 2,
  favoriteFood: 'Steak'
};
const myObjStr = JSON.stringify(myObj);
console.log(myObjStr);
// "{"name":"Sammy","age":6,"favoriteFood":"Tofu"}"
console.log(JSON.parse(myObjStr));
// Object {name:"Sammy",age:6,favoriteFood:"Tofu"}
```

Trabajando con JSON en JS

- **JSON.parse()**: puede tomar una función como segundo argumento que puede transformar los valores de objeto antes de que se devuelvan

```
const user = {
  name: 'Sammy',
  email: 'Sammy@domain.com',
  plan: 'Pro'
};

const userStr = JSON.stringify(user);

JSON.parse(userStr, (key, value) => {
      if (typeof value === 'string') {
        return value.toUpperCase();
      }
      return value;
    });
```

Trabajando con JSON en JS

- **JSON.stringify()**: puede tomar una función como segundo argumento para reemplazar
  - se puede usar para filtrar los valores

```
const user = {
  id: 229,
  name: 'Sammy',
  email: 'Sammy@domain.com'
};
function replacer(key, value) {
      console.log(typeof value);
      if (key === 'email') {
        return undefined;
      }
      return value;
    }
const userStr = JSON.stringify(user, replacer);
// "{"id":229,"name":"Sammy"}"
```